

ASP.NET Core + Entity Framework

Gyakorló Feladat #6

Háziállat-nyilvántartó REST API

Technológia	ASP.NET Core 8, EF Core, SQLite
Adatbázis	2 tábla (Owner → Pet, One-to-Many)
Plusz funkciók	Seed adatok + szűrés / keresés

1. FELADATLEÍRÁS

1.1 Háttér

Egy állatorvosi rendelő szeretné digitalizálni a pácienseinek nyilvántartását. A rendszerben gazdák (owners) és az ő háziállataik (pets) szerepelnek. Egy gazdának több háziállata is lehet, de minden háziállat pontosan egy gazdához tartozik. Az API-nak szűrési és keresési lehetőségeket is kell biztosítania.

1.2 Adatbázis struktúra

Két táblára lesz szükséged — One-to-Many kapcsolattal:

Mező	Típus	Leírás
Gazda tábla (Owners)		
Id	int	Elsődleges kulcs, automatikus növekedés
FirstName	string	Keresztnév (kötelező, max 60 karakter)
LastName	string	Vezetéknév (kötelező, max 60 karakter)
Phone	string?	Telefonszám (opcionális, max 30 karakter)
Email	string?	E-mail cím (opcionális, max 120 karakter)
Háziállat tábla (Pets)		
Id	int	Elsődleges kulcs, automatikus növekedés
Name	string	Háziállat neve (kötelező, max 60 karakter)
Species	string	Faj: "Kutya", "Macska", "Nyúl", "Egyéb" (kötelező)
Breed	string?	Fajta (opcionális, max 80 karakter)
BirthYear	int?	Születési év (opcionális)
OwnerId	int	Külső kulcs az Owners táblára

1.3 API végpontok

Metódus	URL	Leírás
GET	/api/owners	Összes gazda (szűrhető: search)
GET	/api/owners/{id}	Egy gazda lekérdezése háziállataival
POST	/api/owners	Új gazda létrehozása
PUT	/api/owners/{id}	Gazda módosítása
DELETE	/api/owners/{id}	Gazda törlése (csak ha nincs háziállata)
GET	/api/pets	Összes háziállat (szűrhető: species)

Metódus	URL	Leírás
GET	/api/pets/{id}	Egy háziállat lekérdezése
POST	/api/pets	Új háziállat létrehozása
PUT	/api/pets/{id}	Háziállat módosítása
DELETE	/api/pets/{id}	Háziállat törlése

1.4 Szűrési és keresési paraméterek

Végpont	Paraméter	Példa
GET /api/owners	?search=Kovács	Névben keres (kereszt- és vezetéknév)
GET /api/pets	?species=Kutya	Csak kutyák listázása
GET /api/pets	?search=Bodri	Háziállat nevében keres
GET /api/pets	?species=Macska&search=Mi	Kombinálható

1.5 Elvárások

- ASP.NET Core 8 Web API, EF Core SQLite adatbázissal
- Használj DTO osztályokat – ne add vissza közvetlenül az entitásokat
- A GET /api/owners/{id} a gazdát a háziállataival együtt adja vissza
- A Species mező csak "Kutya", "Macska", "Nyúl" vagy "Egyéb" lehet – ellenőrizd
- Gazda törlésénél: ha van hozzá háziállat, 400-as hibával jelezd
- Háziállat létrehozásakor ellenőrizd, hogy az OwnerId létezik-e
- A listák alaphiból névsor szerint legyenek rendezve
- Az adatbázis induláskor töltődjön fel seed adatokkal

2. MEGOLDÁS

2.1 Projekt létrehozása

```
dotnet new webapi -n PetClinicApi
cd PetClinicApi
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
```

2.2 Modell osztályok

Models/Owner.cs

```
namespace PetClinicApi.Models;

public class Owner
{
    public int Id { get; set; }
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string? Phone { get; set; }
    public string? Email { get; set; }

    // Navigációs property
    public List<Pet> Pets { get; set; } = new();
}
```

Models/Pet.cs

```
namespace PetClinicApi.Models;

public class Pet
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Species { get; set; } = string.Empty;
    public string? Breed { get; set; }
    public int? BirthYear { get; set; }

    // Külső kulcs
    public int OwnerId { get; set; }
    public Owner? Owner { get; set; }
}
```

2.3 DTO osztályok

DTOs/OwnerDto.cs

```
namespace PetClinicApi.DTOs;

// Lista nézethez (háziállatok nélkül)
public record OwnerListDto(int Id, string FirstName, string LastName, string?
Phone, string? Email);

// Részletes nézet (háziállatokkal együtt)
public record OwnerDetailDto(
    int Id, string FirstName, string LastName,
    string? Phone, string? Email,
    List<PetSimpleDto> Pets
);

// Létrehozáshoz / módosításhoz
public record OwnerCreateDto(string FirstName, string LastName, string? Phone,
string? Email);
```

DTOs/PetDto.cs

```
namespace PetClinicApi.DTOs;

// Listázáshoz (gazda nevével együtt)
public record PetListDto(
    int Id, string Name, string Species,
    string? Breed, int? BirthYear, string OwnerName
);

// Beágyazva a gazda részletes nézetébe
public record PetSimpleDto(int Id, string Name, string Species, string? Breed,
int? BirthYear);

// Létrehozáshoz / módosításhoz
public record PetCreateDto(
    string Name, string Species, string? Breed, int? BirthYear, int OwnerId
);
```

2.4 DbContext seed adatokkal

Data/PetClinicContext.cs

```
using Microsoft.EntityFrameworkCore;
using PetClinicApi.Models;

namespace PetClinicApi.Data;

public class PetClinicContext : DbContext
{
```

```

    public PetClinicContext(DbContextOptions<PetClinicContext> options) :
    base(options) { }

    public DbSet<Owner> Owners => Set<Owner>();
    public DbSet<Pet> Pets => Set<Pet>();

    protected override void OnModelCreating(ModelBuilder mb)
    {
        mb.Entity<Owner>(e => {
            e.Property(o => o.FirstName).HasMaxLength(60).IsRequired();
            e.Property(o => o.LastName).HasMaxLength(60).IsRequired();
            e.Property(o => o.Phone).HasMaxLength(30);
            e.Property(o => o.Email).HasMaxLength(120);
        });

        mb.Entity<Pet>(e => {
            e.Property(p => p.Name).HasMaxLength(60).IsRequired();
            e.Property(p => p.Species).HasMaxLength(20).IsRequired();
            e.Property(p => p.Breed).HasMaxLength(80);
            e.HasOne(p => p.Owner)
                .WithMany(o => o.Pets)
                .HasForeignKey(p => p.OwnerId);
        });

        // — Seed adatok —
        mb.Entity<Owner>().HasData(
            new Owner { Id = 1, FirstName = "Kovács", LastName = "Anna",
                Phone = "+36 20 123 4567", Email = "anna.kovacs@gmail.com" },
            new Owner { Id = 2, FirstName = "Nagy", LastName = "Béla",
                Phone = "+36 30 987 6543", Email = null },
            new Owner { Id = 3, FirstName = "Horváth", LastName = "Csilla",
                Phone = null, Email = "csilla.h@freemail.hu" }
        );

        mb.Entity<Pet>().HasData(
            new Pet { Id = 1, Name = "Bodri", Species = "Kutya", Breed =
"Labrador",
                BirthYear = 2019, OwnerId = 1 },
            new Pet { Id = 2, Name = "Cirmos", Species = "Macska", Breed =
"Európai rövidszőrű",
                BirthYear = 2021, OwnerId = 1 },
            new Pet { Id = 3, Name = "Rex", Species = "Kutya", Breed = "Német
juhász",
                BirthYear = 2020, OwnerId = 2 },
            new Pet { Id = 4, Name = "Hópihe", Species = "Nyúl", Breed = null,
                BirthYear = 2022, OwnerId = 3 },
            new Pet { Id = 5, Name = "Mici", Species = "Macska", Breed =
"Perzsa",
                BirthYear = 2018, OwnerId = 3 }
        );
    }
}

```

2.5 OwnersController

Controllers/OwnersController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using PetClinicApi.Data;
using PetClinicApi.DTOs;
using PetClinicApi.Models;

namespace PetClinicApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class OwnersController : ControllerBase
{
    private readonly PetClinicContext _db;
    public OwnersController(PetClinicContext db) => _db = db;

    // GET /api/owners?search=Kovács
    [HttpGet]
    public async Task<ActionResult<List<OwnerListDto>>> GetAll([FromQuery] string? search)
    {
        var query = _db.Owners.AsQueryable();

        if (!string.IsNullOrEmpty(search))
            query = query.Where(o =>
                o.FirstName.Contains(search) || o.LastName.Contains(search));

        var owners = await query
            .OrderBy(o => o.LastName).ThenBy(o => o.FirstName)
            .Select(o => new OwnerListDto(o.Id, o.FirstName, o.LastName, o.Phone,
o.Email))
            .ToListAsync();

        return Ok(owners);
    }

    // GET /api/owners/{id}
    [HttpGet("{id}")]
    public async Task<ActionResult<OwnerDetailDto>> GetById(int id)
    {
        var owner = await _db.Owners
            .Include(o => o.Pets)
            .FirstOrDefaultAsync(o => o.Id == id);

        if (owner is null) return NotFound();

        return Ok(new OwnerDetailDto(
```

```

        owner.Id, owner.FirstName, owner.LastName, owner.Phone, owner.Email,
        owner.Pets.Select(p =>
            new PetSimpleDto(p.Id, p.Name, p.Species, p.Breed, p.BirthYear)
        ).ToList()
    ));
}

// POST /api/owners
[HttpPost]
public async Task<ActionResult<OwnerListDto>> Create(OwnerCreatedDto dto)
{
    var owner = new Owner {
        FirstName = dto.FirstName, LastName = dto.LastName,
        Phone = dto.Phone, Email = dto.Email
    };
    _db.Owners.Add(owner);
    await _db.SaveChangesAsync();

    var result = new OwnerListDto(owner.Id, owner.FirstName, owner.LastName,
        owner.Phone, owner.Email);
    return CreatedAtAction(nameof(GetById), new { id = owner.Id }, result);
}

// PUT /api/owners/{id}
[HttpPut("{id}")]
public async Task<IActionResult> Update(int id, OwnerCreatedDto dto)
{
    var owner = await _db.Owners.FindAsync(id);
    if (owner is null) return NotFound();

    owner.FirstName = dto.FirstName;
    owner.LastName = dto.LastName;
    owner.Phone = dto.Phone;
    owner.Email = dto.Email;
    await _db.SaveChangesAsync();
    return NoContent();
}

// DELETE /api/owners/{id}
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var owner = await _db.Owners
        .Include(o => o.Pets)
        .FirstOrDefaultAsync(o => o.Id == id);
    if (owner is null) return NotFound();

    if (owner.Pets.Count > 0)
        return BadRequest(
            $"A gazda nem törölhető, mert {owner.Pets.Count} háziállat
            tartozik hozzá!");
}

```



```

        _db.Owners.Remove(owner);
        await _db.SaveChangesAsync();
        return NoContent();
    }
}

```

2.6 PetsController

Controllers/PetsController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using PetClinicApi.Data;
using PetClinicApi.DTOs;
using PetClinicApi.Models;

namespace PetClinicApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class PetsController : ControllerBase
{
    private readonly PetClinicContext _db;
    private static readonly string[] ValidSpecies = ["Kutya", "Macska", "Nyúl", "Egyéb"];

    public PetsController(PetClinicContext db) => _db = db;

    // GET /api/pets?species=Kutya&search=Bo
    [HttpGet]
    public async Task<ActionResult<List<PetListDto>>> GetAll(
        [FromQuery] string? species,
        [FromQuery] string? search)
    {
        var query = _db.Pets.Include(p => p.Owner).AsQueryable();

        if (!string.IsNullOrEmpty(species))
            query = query.Where(p => p.Species == species);

        if (!string.IsNullOrEmpty(search))
            query = query.Where(p => p.Name.Contains(search));

        var pets = await query
            .OrderBy(p => p.Name)
            .Select(p => new PetListDto(
                p.Id, p.Name, p.Species, p.Breed, p.BirthYear,
                p.Owner!.LastName + " " + p.Owner.FirstName))
            .ToListAsync();
    }
}

```

```

        return Ok(pets);
    }

    // GET /api/pets/{id}
    [HttpGet("{id}")]
    public async Task<ActionResult<PetListDto>> GetById(int id)
    {
        var p = await _db.Pets
            .Include(p => p.Owner)
            .FirstOrDefaultAsync(p => p.Id == id);
        if (p is null) return NotFound();

        return Ok(new PetListDto(p.Id, p.Name, p.Species, p.Breed, p.BirthYear,
            p.Owner!.LastName + " " + p.Owner.FirstName));
    }

    // POST /api/pets
    [HttpPost]
    public async Task<ActionResult<PetListDto>> Create(PetCreateDto dto)
    {
        if (!ValidSpecies.Contains(dto.Species))
            return BadRequest($"Érvénytelen faj! Megengedett értékek:
{string.Join(", ", ValidSpecies)}");

        var owner = await _db.Owners.FindAsync(dto.OwnerId);
        if (owner is null)
            return BadRequest($"Nem létezik gazda {dto.OwnerId} azonosítóval.");

        var pet = new Pet {
            Name = dto.Name, Species = dto.Species,
            Breed = dto.Breed, BirthYear = dto.BirthYear, OwnerId = dto.OwnerId
        };
        _db.Pets.Add(pet);
        await _db.SaveChangesAsync();

        var result = new PetListDto(pet.Id, pet.Name, pet.Species, pet.Breed,
            pet.BirthYear, owner.LastName + " " + owner.FirstName);
        return CreatedAtAction(nameof(GetById), new { id = pet.Id }, result);
    }

    // PUT /api/pets/{id}
    [HttpPut("{id}")]
    public async Task<IActionResult> Update(int id, PetCreateDto dto)
    {
        if (!ValidSpecies.Contains(dto.Species))
            return BadRequest($"Érvénytelen faj! Megengedett értékek:
{string.Join(", ", ValidSpecies)}");

        var pet = await _db.Pets.FindAsync(id);
        if (pet is null) return NotFound();
    }

```

```

        var owner = await _db.Owners.FindAsync(dto.OwnerId);
        if (owner is null)
            return BadRequest($"Nem létezik gazda {dto.OwnerId} azonosítóval.");

        pet.Name = dto.Name; pet.Species = dto.Species;
        pet.Breed = dto.Breed; pet.BirthYear = dto.BirthYear;
        pet.OwnerId = dto.OwnerId;
        await _db.SaveChangesAsync();
        return NoContent();
    }

    // DELETE /api/pets/{id}
    [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id)
    {
        var pet = await _db.Pets.FindAsync(id);
        if (pet is null) return NotFound();

        _db.Pets.Remove(pet);
        await _db.SaveChangesAsync();
        return NoContent();
    }
}

```

2.7 Program.cs

```

using Microsoft.EntityFrameworkCore;
using PetClinicApi.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<PetClinicContext>(opt =>
    opt.UseSqlite("Data Source=petclinic.db"));

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<PetClinicContext>();
    db.Database.Migrate();
}

app.UseSwagger();
app.UseSwaggerUI();
app.MapControllers();
app.Run();

```

2.8 Migráció és futtatás

```
dotnet ef migrations add InitialCreate  
dotnet ef database update  
dotnet run
```

A seed adatok automatikusan betöltődnek: 3 gazda (Kovács Anna, Nagy Béla, Horváth Csilla) és 5 háziállat (Bodri, Cirmos, Rex, Hópihe, Mici). Rögton kipróbálhatod a GET végpontokat!

2.9 Példa request body-k

POST /api/owners – Új gazda

```
{
  "firstName": "Dénes",
  "lastName": "Szabó",
  "phone": "+36 70 333 4444",
  "email": "denes.szabo@gmail.com"
}
```

POST /api/pets – Új háziállat

```
{
  "name": "Füles",
  "species": "Nyúl",
  "breed": "Törpe nyúl",
  "birthYear": 2023,
  "ownerId": 1
}
```

POST /api/pets – Hibás faj (400-at kell kapni)

```
{
  "name": "Tweety",
  "species": "Papagáj",
  "breed": null,
  "birthYear": 2020,
  "ownerId": 2
}
```

2.10 Tesztelési sorrend

1. GET /api/owners – Ellenőrizd a 3 seed gazdát (ABC sorrendben: Horváth, Kovács, Nagy)
2. GET /api/pets – Ellenőrizd az 5 seed háziállatot névsor szerint
3. GET /api/owners/1 – Kovács Anna háziállataival (Bodri és Cirmos)
4. GET /api/pets?species=Kutya – Csak kutyák (Bodri, Rex)
5. GET /api/pets?search=ci – Névben keres (Cirmos, Mici)
6. GET /api/pets?species=Macska&search=Mi – Kombinált szűrés (csak Mici)
7. POST /api/owners – Hozz létre egy új gazdát
8. POST /api/pets – Adj hozzá háziállatot az új gazdához
9. POST /api/pets – Küldj "Papagáj" fajt (400-as hibát kell kapni)
10. POST /api/pets – Küldj nem létező OwnerId-t (400-as hibát kell kapni)
11. DELETE /api/owners/1 – Törlés háziállatokkal (400-as hibát kell kapni)
12. DELETE /api/pets/{id} – Töröld az új háziállatot
13. DELETE /api/owners/{id} – Most már sikerül az új gazda törlése

2.11 Bónusz feladatok

Ha gyorsan végzel, ezekkel mélyítheted a tudásodat!

- Statisztika: GET /api/owners/{id}/stats – a gazda háziállatainak száma faj szerint bontva
- Átadás: PUT /api/pets/{id}/owner/{ownerId} – háziállat átadása másik gazdának
- Szűrés születési évre: GET /api/pets?minBirthYear=2020 – fiatalabb állatok
- Lapozás: GET /api/pets?page=1&pageSize=3

Sok sikert a megvalósításhoz! 🐾

EF Core doku: learn.microsoft.com/ef/core